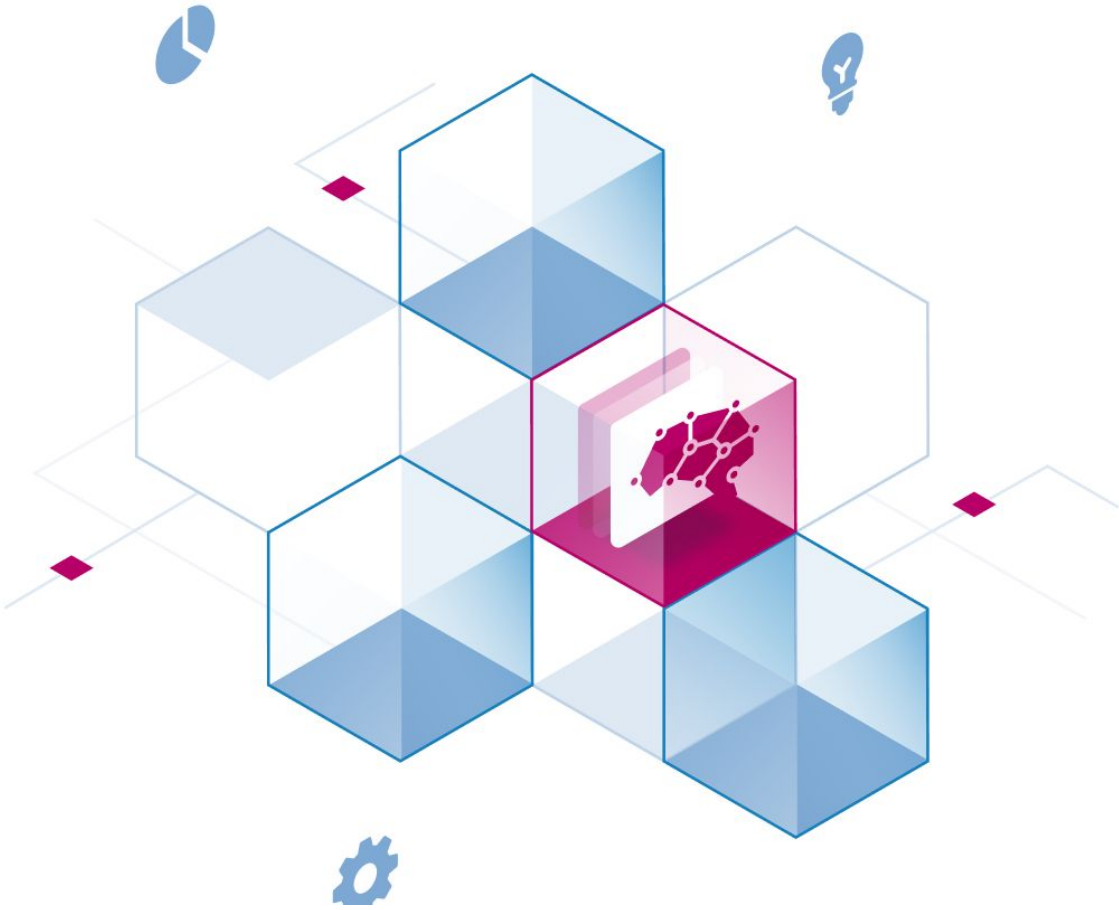
Vom Microservice zum ML-Modell

Eine Odyssee durch moderne Datenformate



Warum dieser Talk?

Wie geht ihr eigentlich mit Schemaveränderungen in Quellsystemen um?

Wir nutzen Iceberg und das verwendet ja unter der Haube Avro und daher ist das kein Problem.

Oh ja, stimmt!

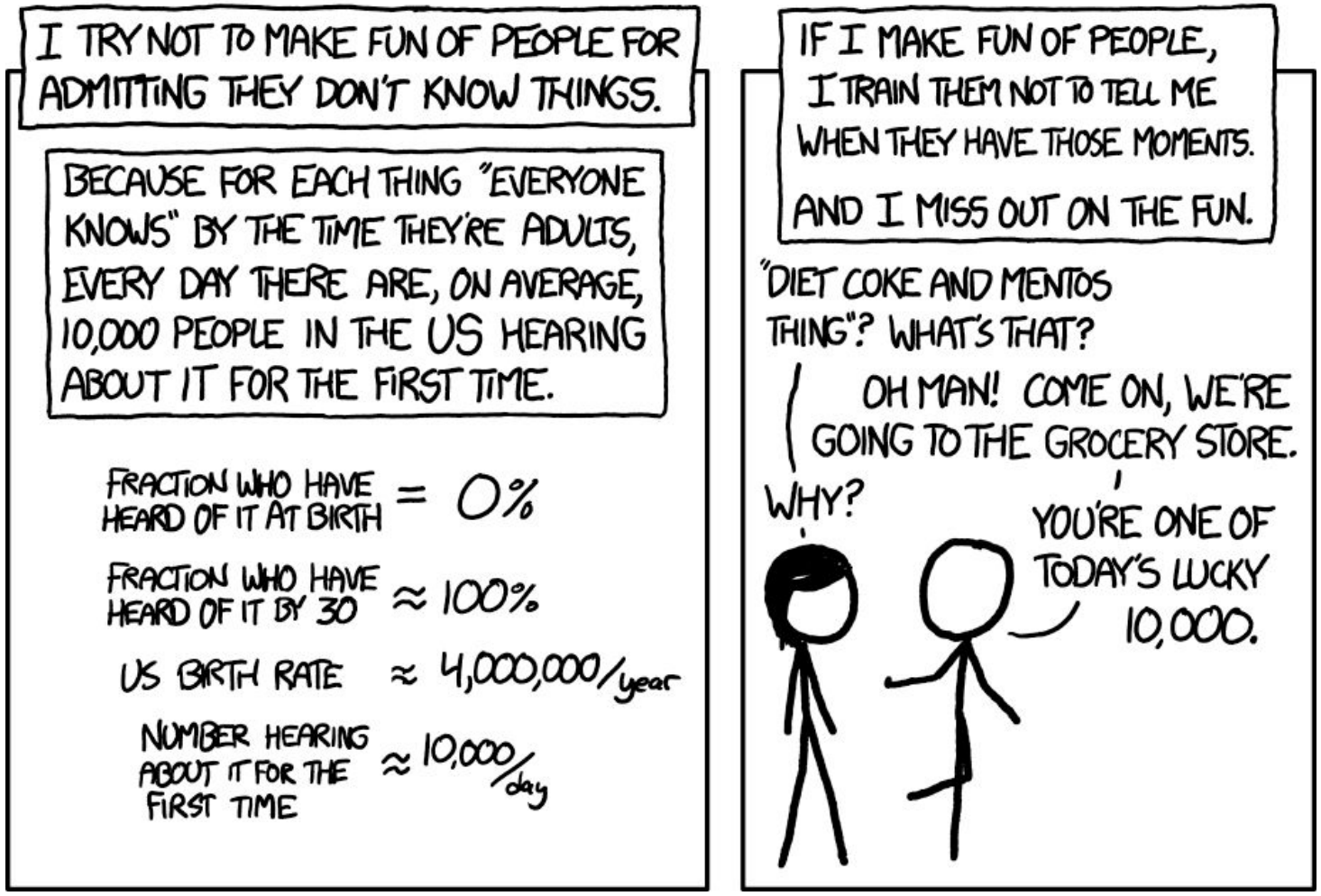
Warum dieser Talk?

Unser Spark Job ist viel zu langsam!

Wie groß sind denn die darunterliegenden Dateien und wie viele sind das?

10k Parquet Dateien, alle so 1-2 KB groß.

Warum dieser Talk?

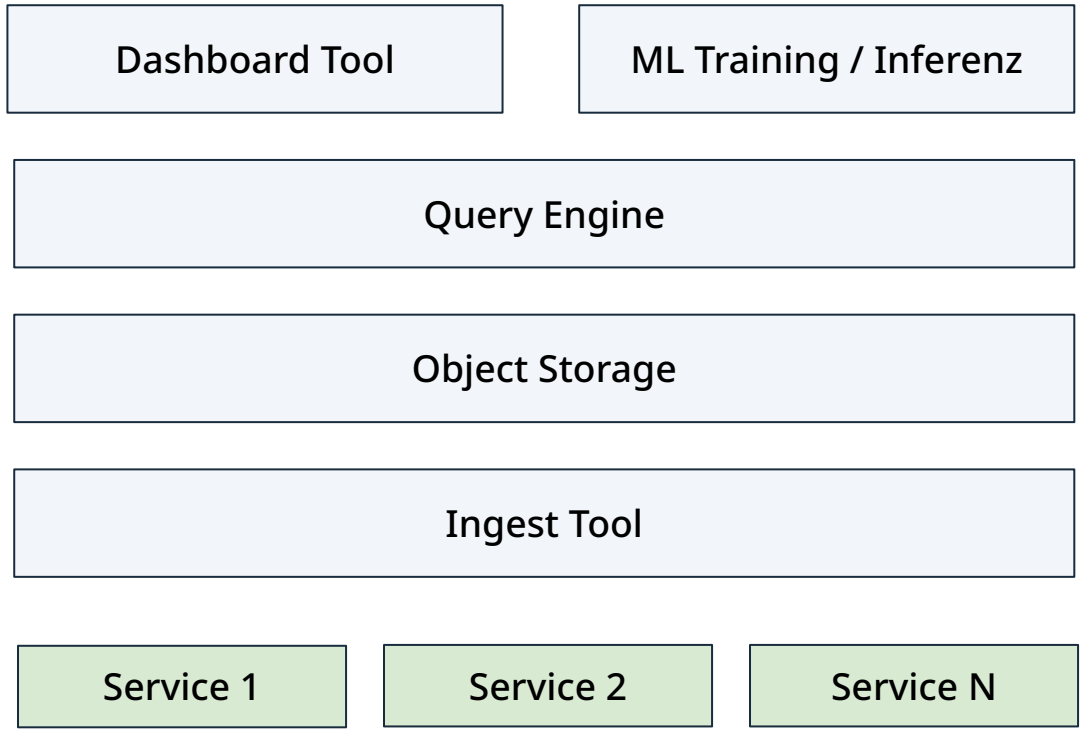


xkcd.com/1053

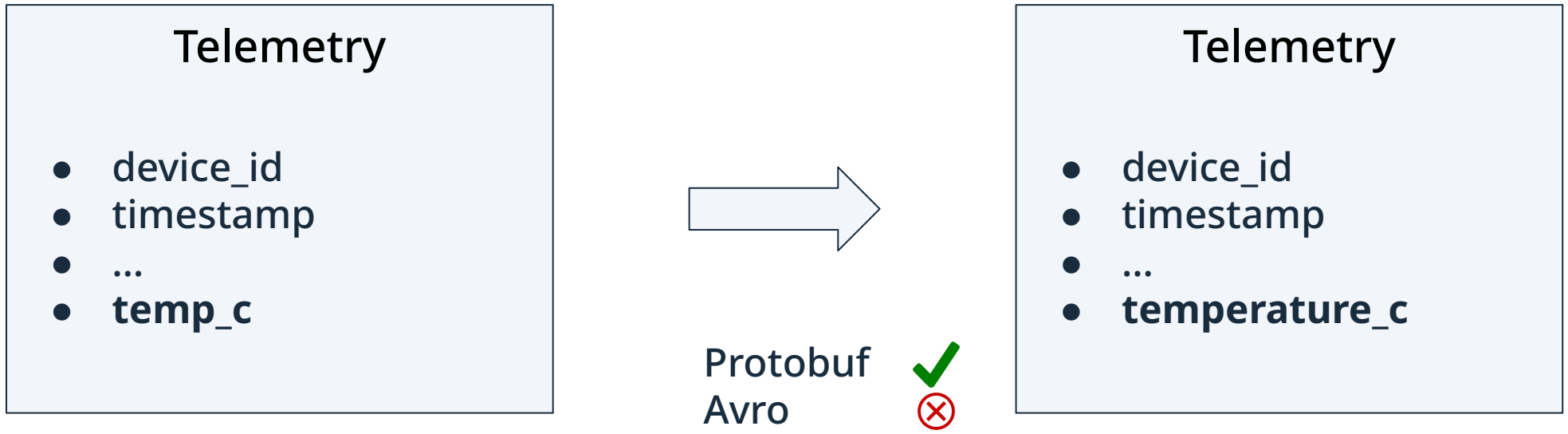
Was erwartet Euch heute?

- Ihr werdet Bytes sehen
- Wir versuchen ein besseres Verständnis von verschiedenen Datenformaten zu entwickeln
- Ein paar Sachen schauen wir uns genauer an als andere

Unsere Datenplattform heute



Wie geht's kaputt?



Was bringt's?

```
SELECT avg(temp_c) FROM telemetry WHERE location = 'hall-a'
```

JSON Lines

- 1.4 GiB fetched
- 11 s CPU

Parquet (partitioniert)

- 6.4 MiB fetched
- 21 ms CPU

Was schauen wir uns an?

2008 Protobuf

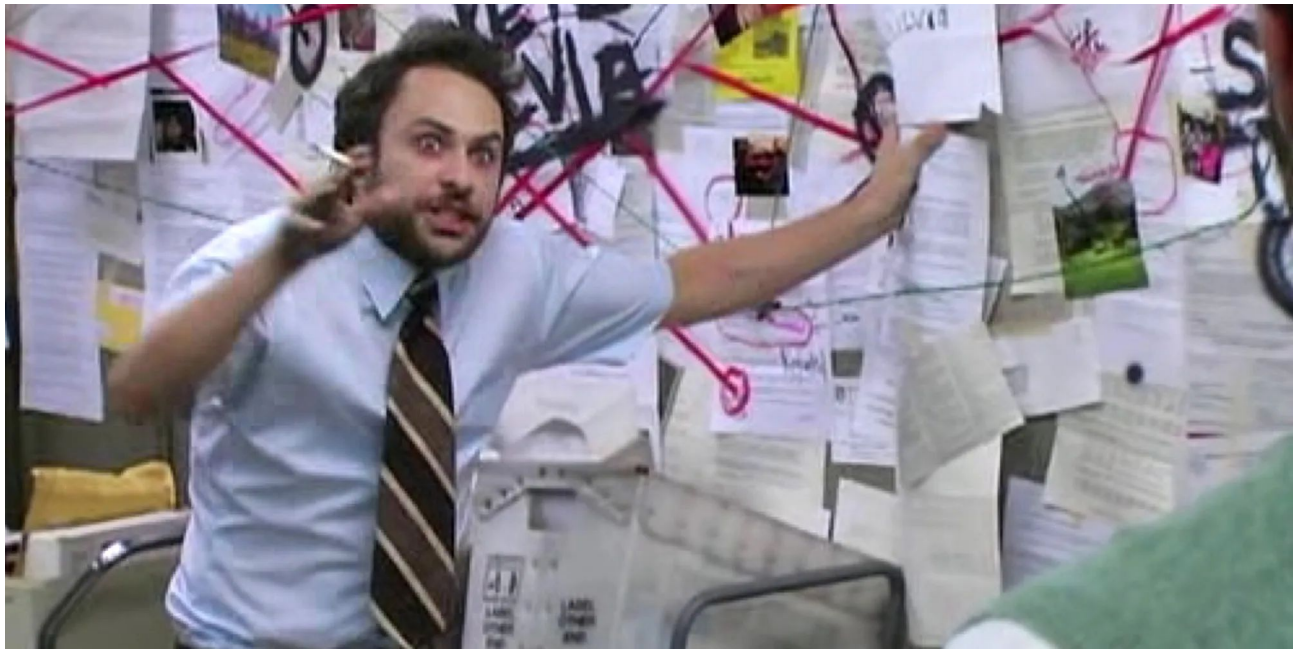
2009 Avro

2013 Parquet

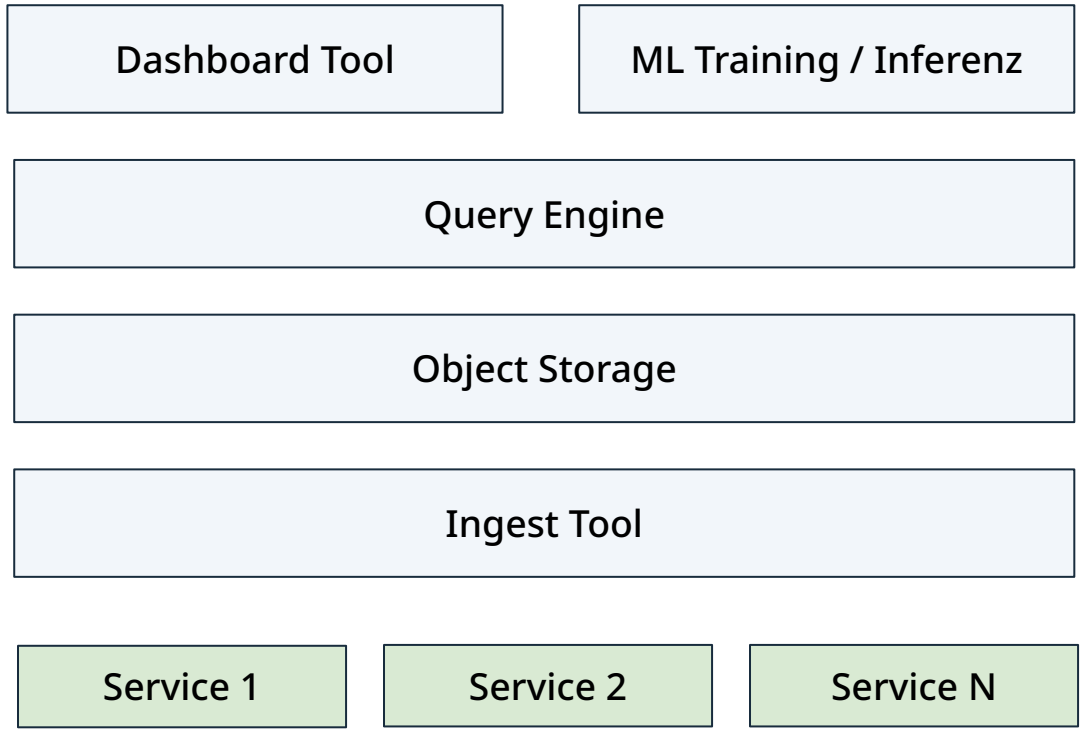
2016 Apache Arrow

2017 Iceberg

2022 Lance



Unsere Datenplattform heute



Protocol Buffers (Protobuf)



- Von Google entwickelt (etwa 2001)
- Anwendungsfall: Daten zwischen Services austauschen
- Hauptkonkurrent damals: XML
- Ähnliches Format: Apache Thrift
- Umfangreiche Verwendung von Code Generatoren

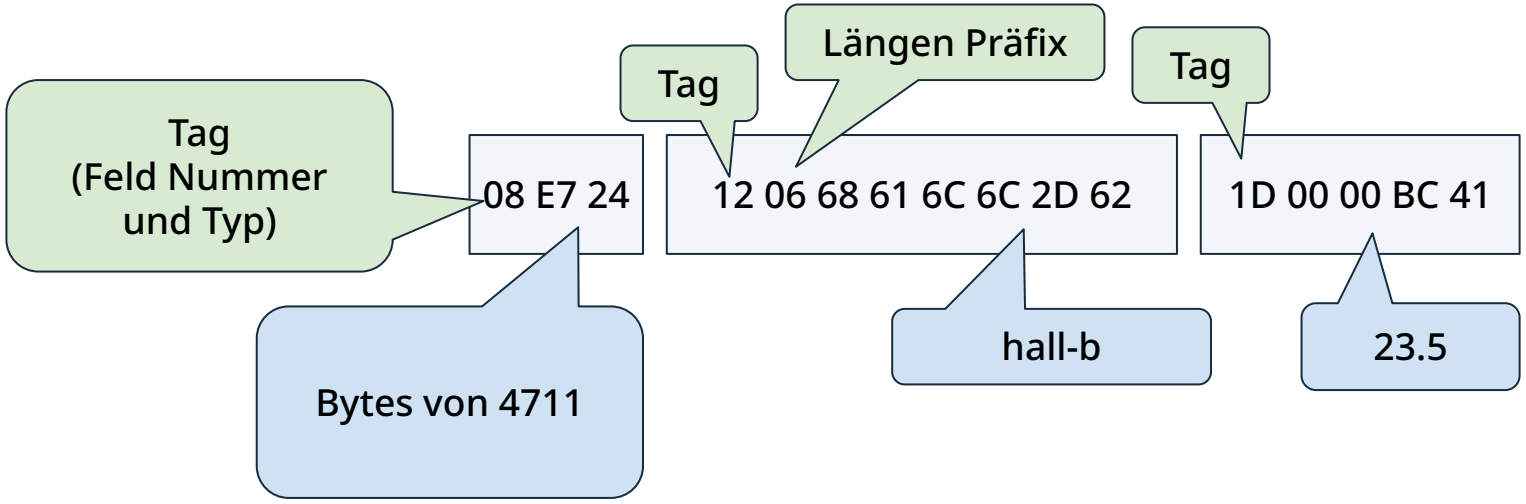
Protocol Buffers (Protobuf)



```
syntax = "proto3";

message Reading {
  uint64 device_id = 1;
  string location = 2;
  float temp_c = 3;
}
```

```
{
  "device_id": 4711,
  "location": "hall-b",
  "temp_c": 23.5
}
```

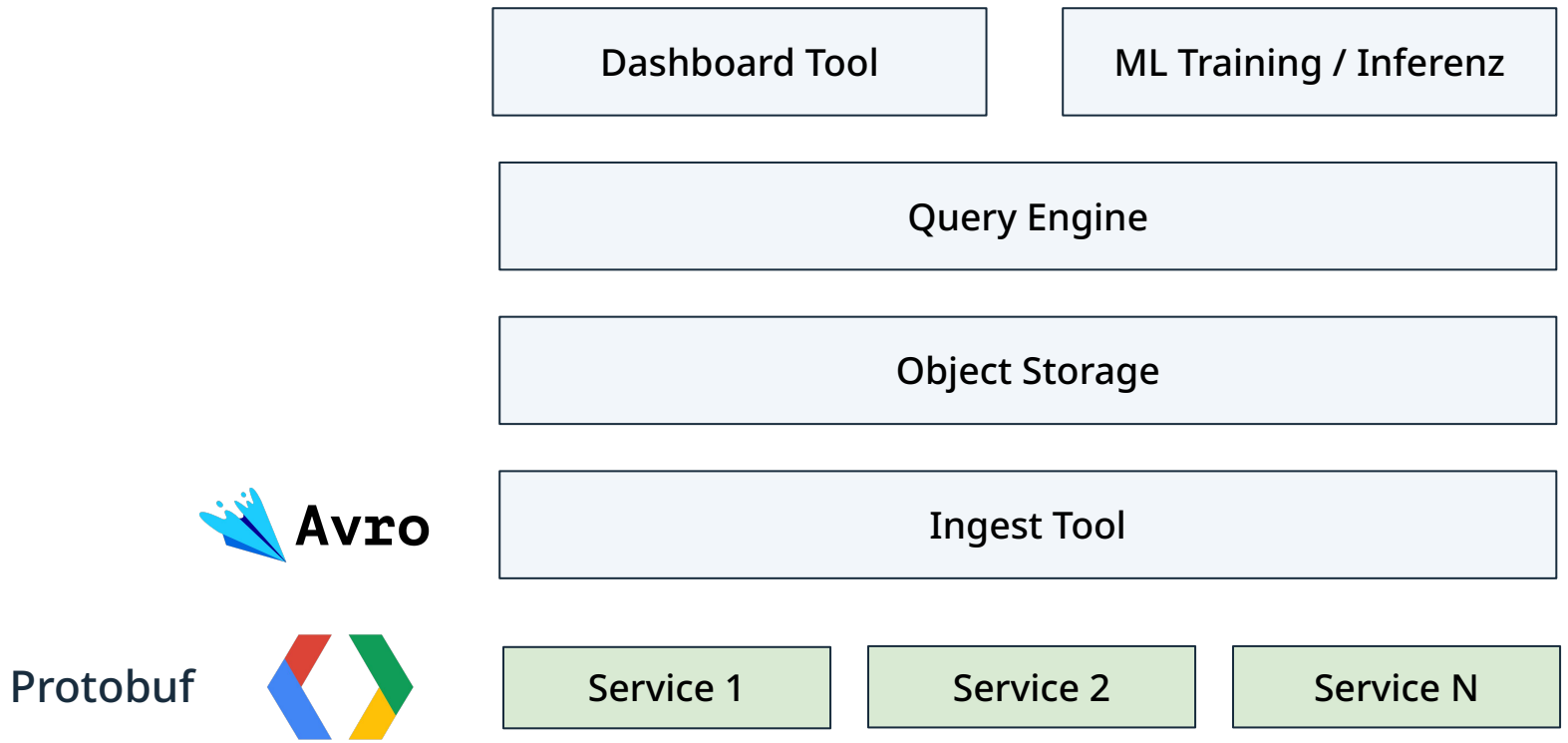


Was bekomme ich, wenn ich die Schema Datei nicht mehr habe?



```
1: 4711
2: "hall-b"
3: 0x41bc0000
```

Unsere Datenplattform heute



Apache Avro



- Initial für Hadoop entwickelt (2009)
- Anwendungsfall: Datenaustausch für Hadoop
- Schema wird mitgeliefert
- Code Generatoren werden nicht benötigt

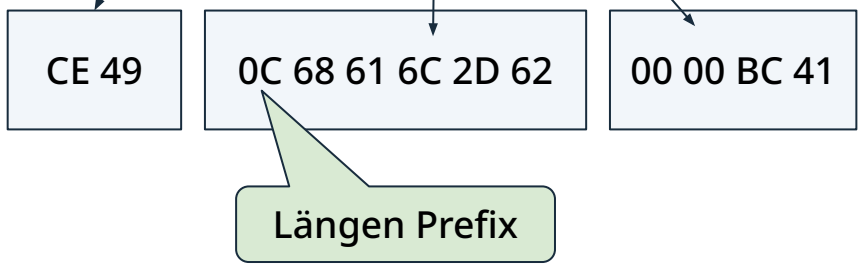
```
{
  "type": "record",
  "name": "Reading",
  "namespace": "conference.avro",
  "fields": [
    {
      "name": "device_id",
      "type": "long"
    },
    {
      "name": "location",
      "type": "string"
    },
    {
      "name": "temp_c",
      "type": "float"
    }
  ]
}
```

Apache Avro - ein einzelner Record

Avro

```
{
  "type": "record",
  "name": "Reading",
  "namespace": "conference.avro",
  "fields": [
    {"name": "device_id","type": "long"},
    {"name": "location","type": "string"},
    {"name": "temp_c","type": "float"}
  ]
}
```

```
{
  "device_id": 4711,
  "location": "hall-b",
  "temp_c": 23.5
}
```



Apache Avro - Wo lebt das Schema?

Avro

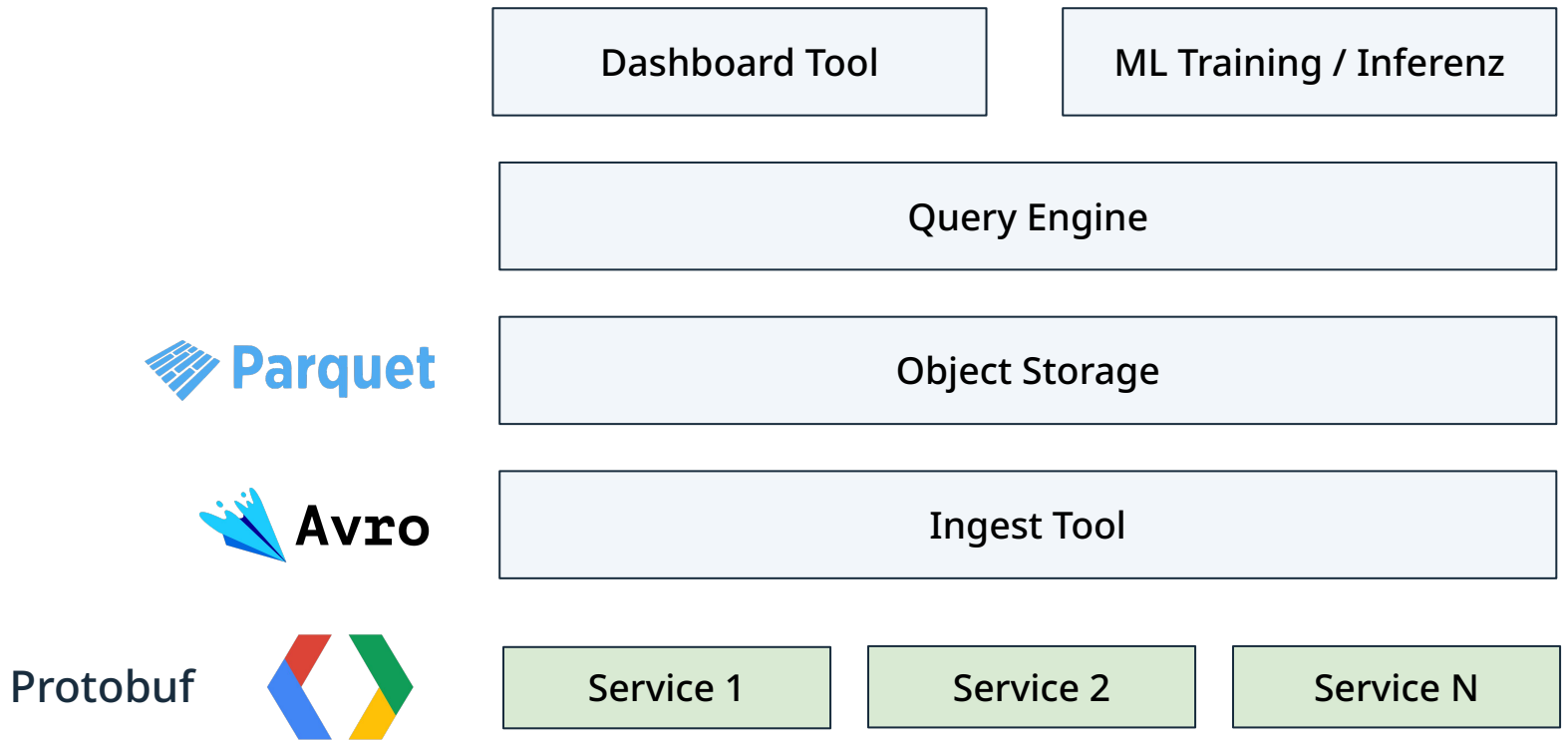
Container File

- Datei Header
  - Magic Byte
  - Komplettes Schema
  - Codec
  - Sync Marker
- Datenblöcke
  - Anzahl / Größe
  - Daten
  - Sync Marker

Schema Registry

- Magic Byte
- Schema ID
- Record

Unsere Datenplattform heute

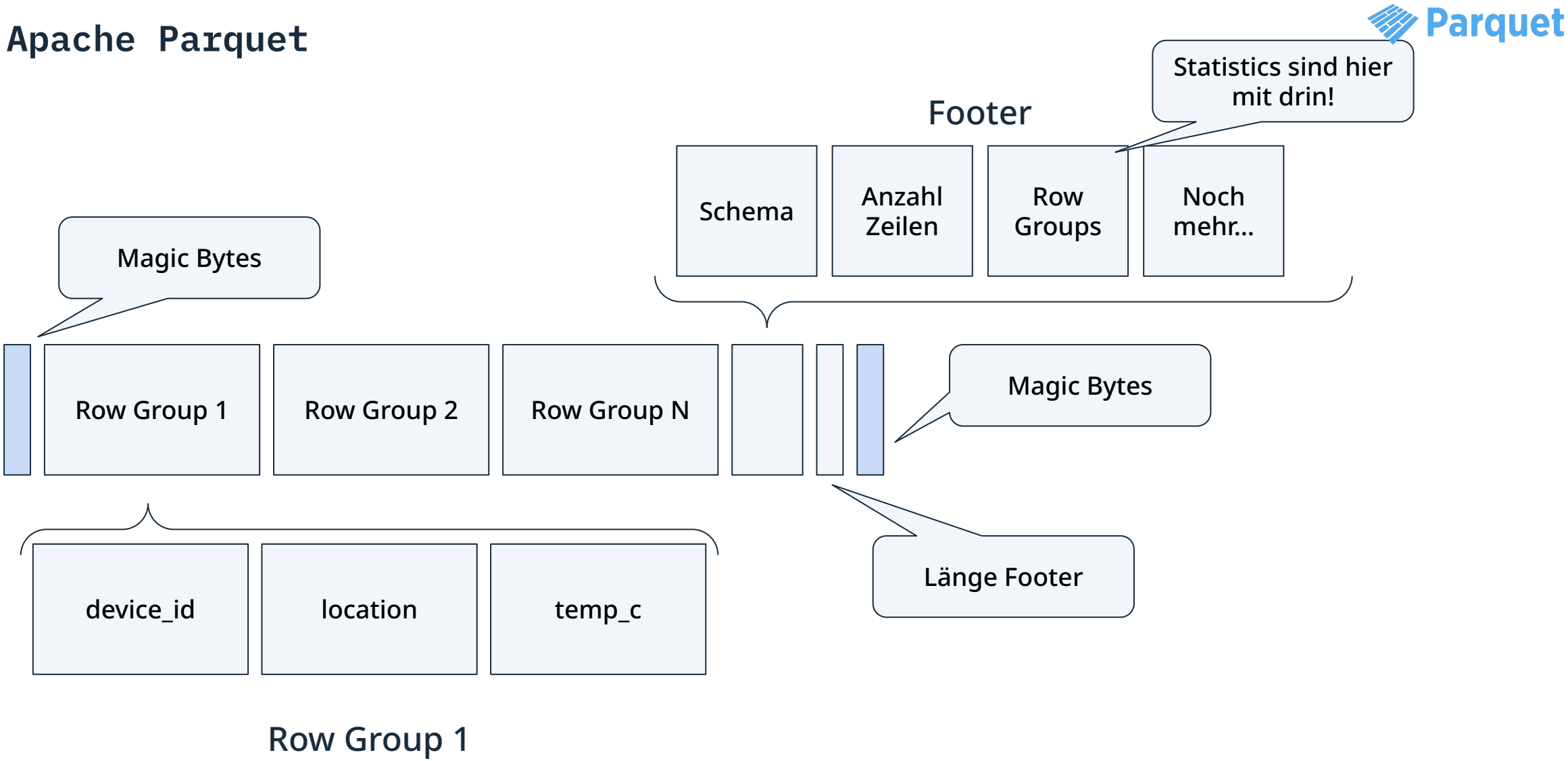


Apache Parquet



- Von Google's Dremel inspiriert, von Twitter/Cloudera entwickelt und 2013 released
- Anwendungsfall: Ad Hoc Analyse Fragestellungen
- Spaltenorientiertes Format
- Fortgeschrittene Datenkompression (Bit-Packing, Run Length Encoding (RLE))

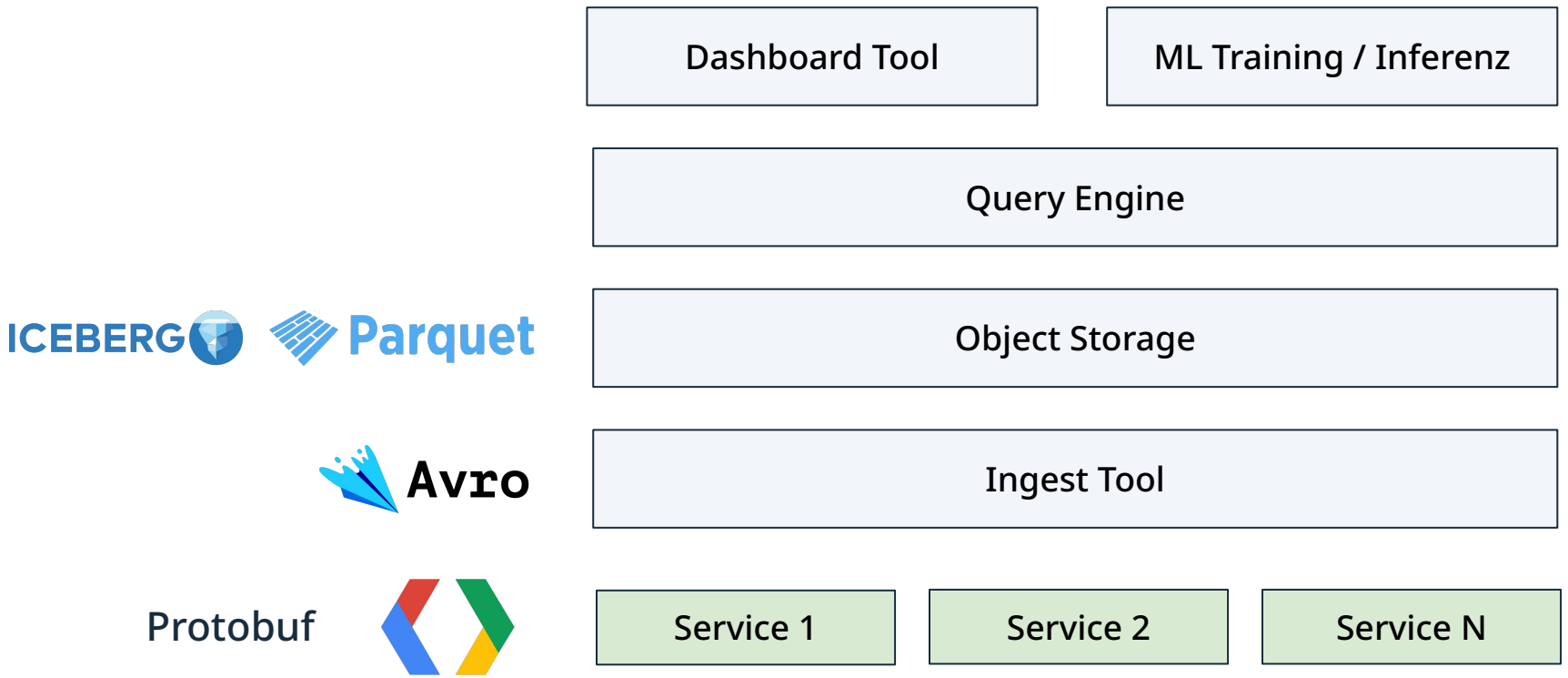
Apache Parquet



Mal zwischendurch: Schema Evolution

	Protobuf	Avro	Parquet	Iceberg (Spoiler)
Feld hinzufügen (mit Default)	✓	✓	✓	✓
Feld hinzufügen (kein Default)		⊗		✓
Feld entfernen	✓	✓		✓
Feld umbenennen	✓	⊗		✓
Int -> Long	✓	✓	✓	✓
Spalten neu ordnen	✓	✓	✓	✓

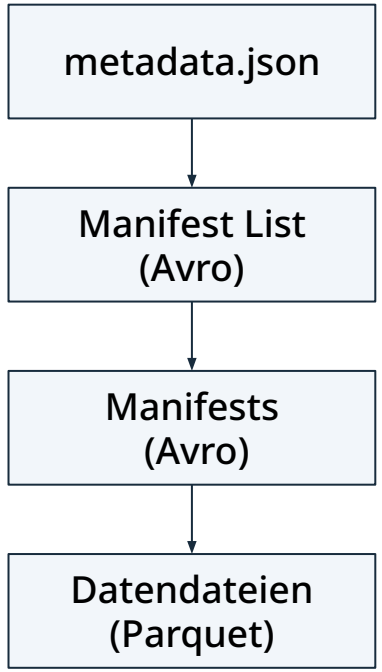
Unsere Datenplattform heute



Apache Iceberg

ICEBERG

- Initial von Netflix entwickelt (2017)
- Anwendungsfall: Tabellen auf Object Storage, mit Transaktionen
- Kein Dateiformat sondern ein Tabellenformat: Metadaten über Parquet Dateien
- Spalten werden über Feld-IDs zugeordnet, nicht über Namen

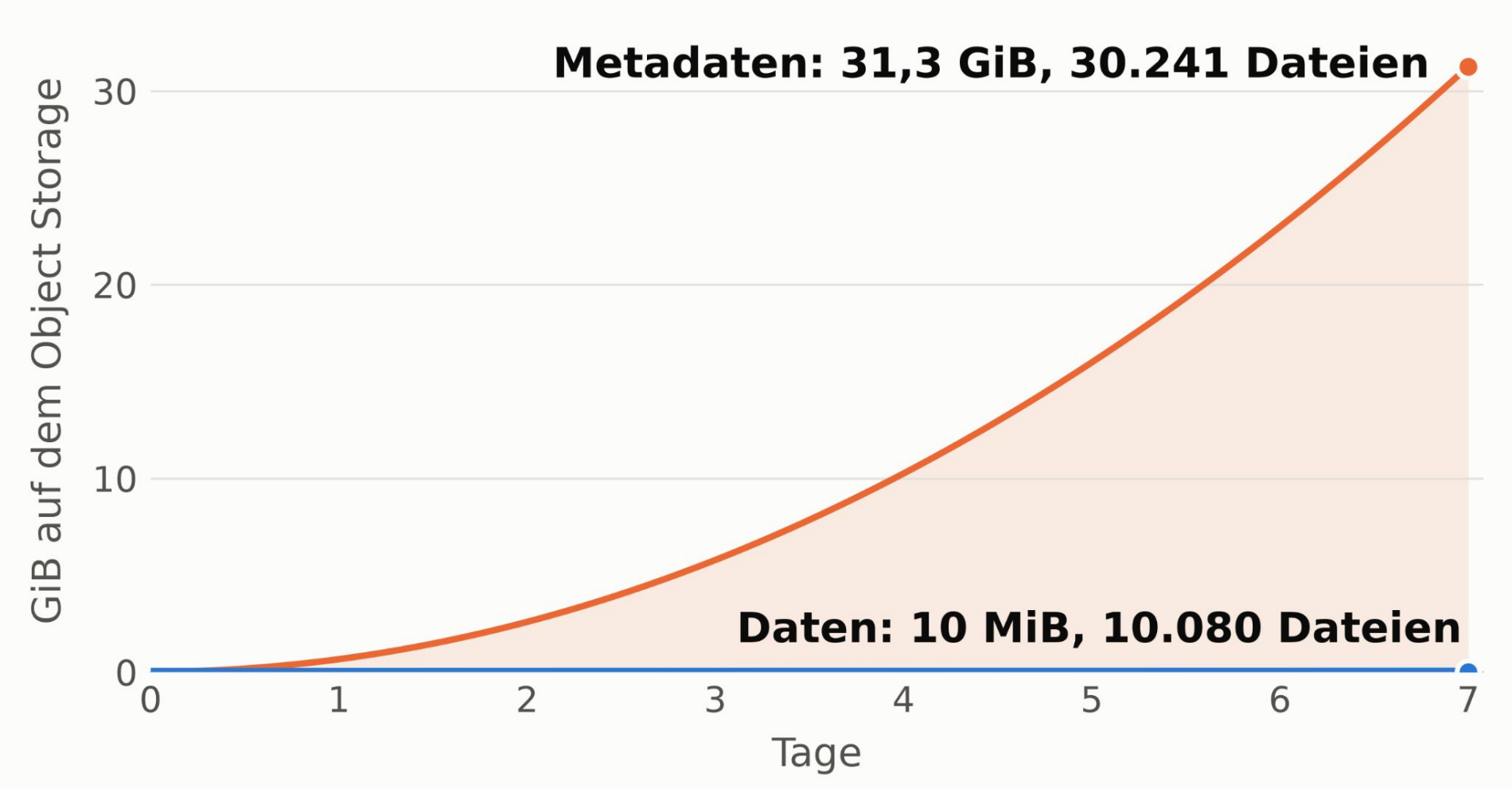


Apache Iceberg

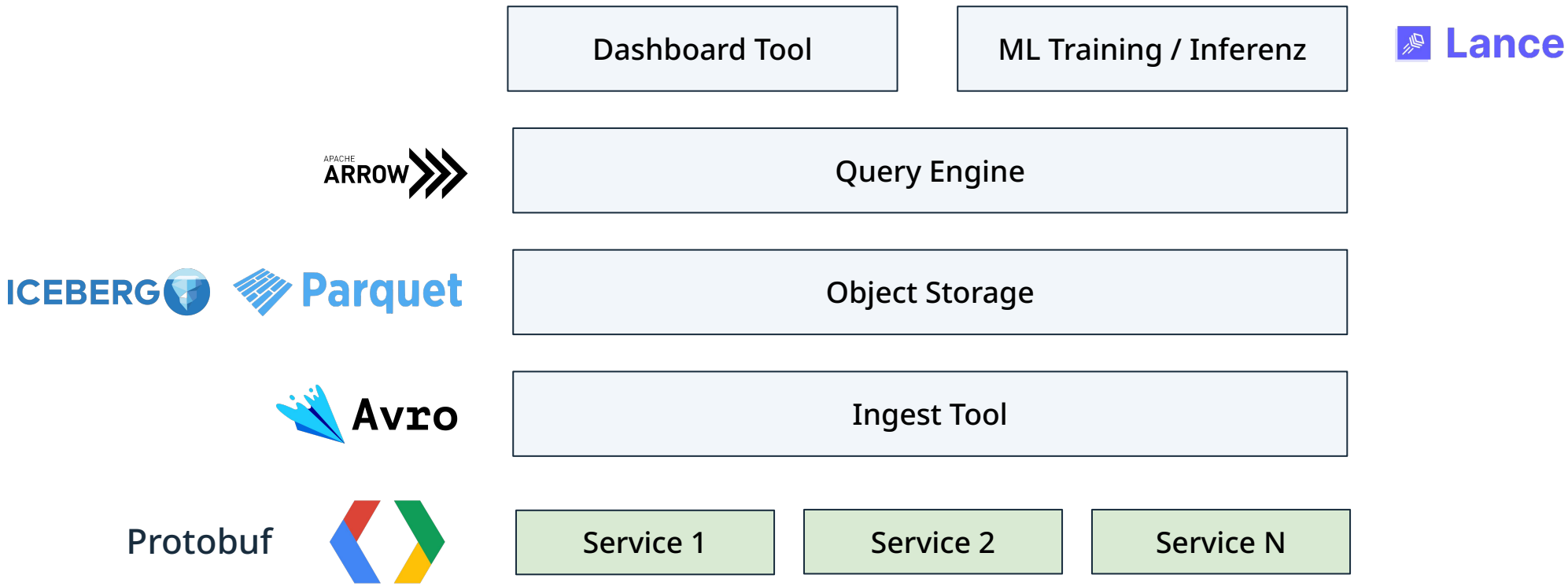
ICEBERG

Änderung (10 Mio. Zeilen)	Daten geschrieben	Metadaten	Zeit
Spalte humidity hinzufügen	0 B	3 KiB	5 ms
temp_c -> temperature_c	0 B	3 KiB	5 ms
device_id von int zu long	0 B	4 KiB	5 ms
firmware_version entfernen	0 B	5 KiB	5 ms
Spalten umsortieren	0 B	5 KiB	5 ms

Apache Iceberg - Ein Commit pro Minute ohne Aufräumen

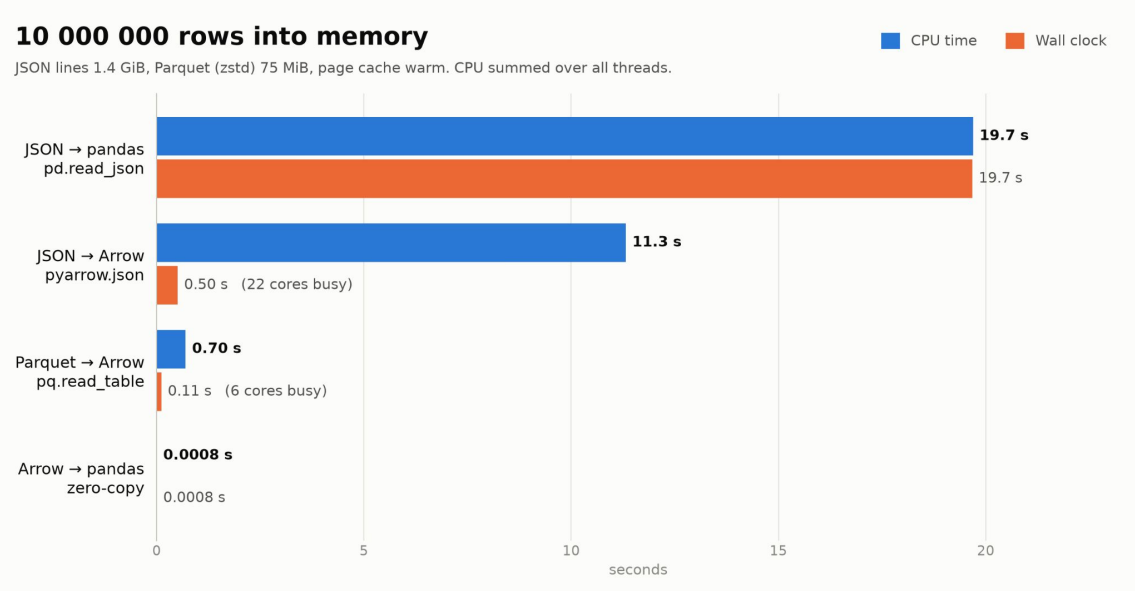


Unsere Datenplattform heute



Apache Arrow

- Entwickelt als Teil der Apache Software Foundation (2016)
- Anwendungsfall: Analytics, aus Arbeitsspeicher Sicht
- Ermöglicht die Verwendung von Daten über Anwendungsgrenzen hinweg



Lance

Lance

- Entwickelt von LanceDB (2022)
- Anwendungsfall: ML-Training und Vektorsuche
- Keine Row Groups: kleine Chunks pro Spalte
- Feld IDs mit Versionierung

1000 zufällige Zeilen aus 10 Mio.

	Parquet	**Lance**
Wall	2471 ms	194 ms
CPU	7757 ms	312 ms

Wrap up

Wie geht ihr eigentlich mit Schemaveränderungen in Quellsystemen um?

Kommt drauf an über welche Änderungen wir reden - was meinst Du denn konkret?

Öhhhmm...

Datenformate geben uns die Werkzeuge um solche Domänenfragen zu beantworten - nicht jeder Fall kann gleich behandelt werden. Wie man damit umgeht, ist Teil der Datenmodellierung.

Wrap up

Unser Spark Job ist viel zu langsam!

Wie groß sind denn die darunterliegenden Dateien und wie viele sind das?

10k Parquet Dateien, alle so 1-2 KB groß.

Compaction und im Falle von Iceberg Snapshots und Metadaten aufräumen

Wrap up

- Wir haben Bytes gesehen
- Wir haben hoffentlich einen besseren Überblick und ein besseres Verständnis gängiger Datenformate
- Wir haben uns ein paar Sachen zu genau angesehen
- Wir haben uns ein paar Sachen nicht genau genug angesehen